SERVICE MESH RUNBOOK — NX-OPS-MESH-010

Service Mesh — mTLS, Traffic Management & Resilience

Nexlify Technologies | Mesh: Istio 1.22 (ambient + sidecar hybrid) | ap-south-1

1. Mesh Adoption Status

Nexlify runs Istio 1.22 in a hybrid model: ambient mode for the majority of namespaces (waypoint proxies for L7), with classic sidecars retained for a small set of latency-sensitive services that require full Envoy feature coverage. The mesh control plane (istiod) is deployed in the platform-system namespace with high availability. Automatic sidecar injection is enabled via namespace labels; ambient enrollment is controlled by the istio.io/dataplane-mode label. All production namespaces are mesh-enabled; non-prod namespaces may opt in. The mesh is the source of truth for service-to-service authentication and most traffic policies; Kong and NGINX handle north-south concerns.

2. Mutual TLS (mTLS)

STRICT mTLS is enforced mesh-wide via a PeerAuthentication resource in the istio-system namespace. Namespaces that have not yet completed migration may temporarily use PERMISSIVE mode; a Kyverno policy alerts on any new PERMISSIVE setting. Certificates are issued by the Istio CA (or by an external CA integrated via the Istio CSR API). Certificate rotation is automatic (default 24h lifetime). Destination rules and PeerAuthentication work together to ensure that plaintext traffic is rejected. For services that must accept traffic from outside the mesh (e.g., legacy clients), a dedicated Gateway with TLS termination is used rather than weakening the mesh policy.

3. Weighted Traffic & Canary Releases

VirtualService and DestinationRule resources implement weighted traffic splitting. A typical canary workflow shifts 5% → 25% → 50% → 100% of traffic to a new revision while monitoring error rate and latency. Argo Rollouts integrates with Istio to automate the analysis and promotion. Subset labels (version: v1, version: v2) are required on pods. Mirror (shadow) traffic is supported for dark launches but must not be used on routes that mutate data. Traffic policies are reviewed in the same PR that introduces a new Deployment revision.

4. Retries, Timeouts & Circuit Breaking

Default retry policy for idempotent methods (GET, HEAD, OPTIONS) is 2 attempts with a 25ms base per-try timeout and exponential backoff. Non-idempotent methods do not retry by default. Route-level timeouts are set to 15 seconds unless the service explicitly requires longer (documented exception). Circuit breaking is configured via DestinationRule outlier detection: consecutive 5xx errors, baseEjectionTime 30s, maxEjectionPercent 50. Connection pool settings limit max outstanding requests per connection. These defaults are applied via a mesh-wide Sidecar or Telemetry resource and can be overridden per-service when necessary. Teams are expected to tune timeouts based on their p99 latency plus a safety margin.

5. Authorization Policies

AuthorizationPolicy resources implement service-to-service and end-user authorization. Policies are namespace-scoped by default; a mesh-wide deny-all is not used because it complicates onboarding. Instead, each namespace starts with an allow-same-namespace rule and adds explicit principals for cross-namespace callers. JWT validation can be performed at the mesh layer for inbound traffic that carries end-user tokens. Workload identity (SPIFFE IDs derived from service accounts) is the primary principal for service-to-service calls. Policy changes are audited via the Istio access log and the Kubernetes audit log.

6. Observability & Operations

Kiali is deployed for topology visualisation; Grafana dashboards cover request rate, error rate, and latency (RED metrics) per service. Distributed tracing uses Jaeger with sampling rate 1% in production and 10% in non-prod. Mesh upgrades follow the same cadence as EKS control plane upgrades and are first validated in non-prod. When a waypoint or sidecar becomes unhealthy, the Platform team checks istiod logs, proxy status (istioctl proxy-status), and the relevant AuthorizationPolicy or PeerAuthentication for misconfiguration.

Owner: Service Mesh Squad | Slack: #service-mesh | Rev 1.9 | 2026-07-18